

DIT Forms Project Documentation

This document outlines the specifications for a custom web application for managing **editable forms**, **student submissions**, **file storage**, and **billing** for HandOuts Tracker items.

Project Goals

The application enables:

- **Editable forms** created and managed by administrators
- **Student submissions** collected without requiring login (admin-only access to view)
- **HandOuts Tracker integration** for automatic invoice generation
- **File storage** in Cloudflare R2 with permission-based access control (admin + class representatives)

Access Control (RBAC)

Role	Permissions
Admin	Full access to students, forms, submissions, invoices, payments, and file download/view
Class Rep	Access to submissions and file downloads only for their assigned <code>programClassId</code> + <code>termId</code>

RBAC Rules:

- Admins can access everything
- Class reps can only view submissions and download files within their assigned scope

Technology Stack

The system uses **Option A** architecture:

Component	Technology
Frontend	HTML + CSS + JavaScript (plain pages)

Component	Technology
Backend	FastAPI (Python)
Database	MongoDB
File Storage	Cloudflare R2
Authentication	JWT
Admin UI	Calls FastAPI APIs; renders data in tables/lists

High-Level Components

Core Services

1. Auth Service (FastAPI)

- Handles user authentication via JWT

2. Form Engine

- Manages form definitions and immutable form versions
- Stores schema as JSON describing fields

3. Submission Service

- Validates base fields
- Stores submission answers
- Handles R2 upload signing and metadata

4. Billing Service

- Processes HandOuts Tracker submissions only
- Auto-generates invoices
- Tracks payment status

5. RBAC Enforcement

- Applied to every admin/class-rep endpoint
- Applied to file download endpoints

MongoDB Schema Overview

A) Users Collection

Fields:

- `_id` (ObjectId)
- `email` (unique)
- `passwordHash`
- `role` (admin | class_rep)
- `assignedClassTerms` (array of { `programClassId`, `termId` } — class rep only)
- `status` (active/disabled)

B) Students Collection

Fields:

- `_id` (ObjectId)
- `programClassId`
- `termId`
- `fullName`
- `idNumber` (e.g., 01240001C)
- `createdAt`, `updatedAt`

Unique Index: { `programClassId`: 1, `termId`: 1, `idNumber`: 1 }

C) Form Definitions Collection

Fields:

- `_id` (ObjectId)
- `name` (e.g., "End of Sem IC List")
- `programClassId`
- `termId`
- `purpose` (free text category, e.g., "Entrepreneurship handout")
- `courseId` (optional)
- `status` (active/inactive)

- createdAt, updatedAt

D) Form Versions Collection

Fields:

- _id (ObjectId)
- formDefinitionId
- versionNumber
- schema (JSON describing fields)
- isActive (recommended)
- createdAt

Schema JSON should define:

- Base fields: Name, ID Number (required)
- Dynamic fields: text, select, date, textarea, number, file
- Required field indicators
- File field keys for upload mapping

E) Form Submissions Collection

Fields:

- _id (ObjectId)
- formVersionId
- formDefinitionId
- programClassId, termId (copied from form definition for stability)
- submittedAt
- status (submitted/processing/closed — optional)
- studentMatch :
 - matchedStudentId (nullable)
 - idNumberSnapshot
 - fullNameSnapshot
- answers (JSON; includes file field answers as fileIds or r20objectKeys)

F) Submission Files Collection (Metadata Only)

Fields:

- `_id` (ObjectId)
- `submissionId`
- `fieldkey` (which form file input)
- `programClassId`, `termId` (copied for quick permission checks)
- `fileName` (original)
- `contentType`
- `sizeBytes`
- `r2Key` (object path/key in R2)
- `uploadedAt`

G) Handout Orders Collection

One document per HandOuts Tracker submission. Created automatically.

Fields:

- `_id` (ObjectId)
- `formSubmissionId` (unique)
- `programClassId`, `termId`
- `student` :
 - `matchedStudentId` (nullable)
 - `fullNameSnapshot`
 - `idNumberSnapshot`
- `givenOutAt` (from submission)
- `invoice` :
 - `invoiceStatus` (unpaid | paid | partially_paid)
 - `currency` (optional)
 - `totalAmount`
- `lines` (all in one document):
 - `[{ courseId, handoutItemId, qty, unitPrice, lineTotal }]`
- `createdAt`

H) Payments Collection

Separate from `handout_orders`.

Fields:

- `_id` (ObjectId)
- `handoutOrderId` (Or `formSubmissionId`)
- `amount`
- `currency` (optional)
- `method` (cash/bank/etc.)
- `reference` (optional)
- `paidAt`
- `receivedByUserId` (optional)

HandOuts Tracker Rules

A) One Invoice Per Submission

Each HandOuts Tracker submission generates exactly **one** `handout_orders` document with `invoice.totalAmount` .

B) Track When Handouts Are Given

- Store `givenOutAt` on the `handout_orders` document
- Source this from the submission's field

C) "One Student Can Only Buy One Handout Per Course" (Per Term)

Rule: For the same `programClassId` + `termId` , a student may have only one handout for a given `courseId` .

Enforcement: Application-level validation at submit time:

1. Resolve student match using `{ programClassId, termId, idNumber }`
2. Check existing `handout_orders` for `(studentId, termId, courseId)`
3. Reject duplicates (recommended) or reject that line item

Form Engine (Editable by Admins)

- Admin creates a form definition
- Admin edits by creating a **new form version** (no overwriting old schema)
- When rendering for submission, the **active version** is used
- Submissions reference the exact `formVersionId`

File Uploads (Cloudflare R2, Admin-Controlled Viewing)

Upload Flow

1. Client requests a **presigned upload URL** from FastAPI for a specific submission and `fieldKey`
2. Browser uploads directly to R2
3. FastAPI stores metadata in `submission_files` **after confirmation** (or stores metadata before and marks complete)

View/Download Flow

1. Endpoint checks **RBAC**:
 - `admin` → always allowed
 - `class_rep` → allowed only if submission belongs to their assigned `{programClassId, termId}`
2. FastAPI returns a **presigned R2 download URL** or **streams the file**

FastAPI Endpoints (MVP List)

Auth

- POST `/auth/login`
- GET `/auth/me`

Admin/Class Rep Shared (Permission-Protected)

- GET `/students?programClassId=&termId=&q=`
- GET `/submissions?programClassId=&termId=&formId=&studentId=&from=&to=`
- GET `/submissions/{submissionId}`
- GET `/files/{fileId}/download`

Admin-Only

- POST /students/import (CSV/XLSX)
- POST /students
- PATCH /students/{id}
- POST /forms
- GET /forms
- POST /forms/{formId}/versions
- GET /forms/{formId}
- GET /handout-orders?programClassId=&termId=&studentId=
- POST /payments (record payment; links to handoutOrderId)
- GET /students/{studentId}/balance

Public Submission Endpoints

- POST /public/forms/{formId}/submit
- POST /submissions/{submissionId}/files/presign

Frontend Pages (HTML/CSS/JS)

- login.html
- admin/dashboard.html
- admin/students.html (+ import UI)
- admin/forms.html
- admin/form-builder.html (creates versions)
- admin/submissions.html (filters)
- admin/submission-detail.html (answers + file links)
- admin/handout-orders.html
- admin/payments.html
- admin/balance.html (optional)
- public/submit-form.html (renders schema and submits)

MVP Build Order

1. **Auth** (admin + class_rep) + JWT + RBAC checks

2. **Students** import + CRUD
3. **Form builder** v1 (schema JSON + form versioning)
4. **Public submission** create + submission listing/detail
5. **R2 presigned upload** + admin/class_rep file download with RBAC
6. **HandOuts Tracker** extraction + `handout_orders` auto-create + invoice totals
7. **Payments** entry + invoice paid status
8. **Enforce** "one handout per course per student per term"

Decision Confirmed: Per-Term Enforcement

You've selected option (A): Per-term only enforcement.

This means the validation logic for "one handout per course per student" will check within the same `programClassId` + `termId` scope only, allowing the same student ID to purchase the same course handout in different terms.

Updated Validation Logic

At submit time for a HandOuts Tracker submission:

1. **Resolve student match** using `{ programClassId, termId, idNumber }`
2. **Check existing handout_orders** for `(studentId, termId, courseId)`
3. **Reject duplicates** within that term (or reject that specific line item if the submission contains multiple courses)

This approach is **recommended** because student IDs may repeat across different academic terms, and it's reasonable to allow re-enrollment in the same course across terms.

Submit-Time Validation Logic (Final)

For each line in the submission:

- **Extract** `courseId`, `handoutItemId`, `qty`
- **Resolve** `studentId` from `idNumber` using `{ programClassId, termId }`
- Query `handout_orders` **where**:
 - `{ studentId, termId, "lines.courseId": courseId }`

- **If match** exists:
 - Reject submission or reject that line item (application choice)

This validation is now **final** and should be implemented in the **Submission Service** (step 8 of the MVP Build Order).